Case Study: Database Normalization from 0NF to BCNF

This document walks through the step-by-step normalization process of a Human Resources (HR) and Access Control database. We start with a single unnormalized **Universal Table (0NF)** and break it down until we reach **Boyce-Codd Normal Form (BCNF)**.

---

0. Initial State: Unnormalized Form (0NF)

In **0NF**, everything is stored inside a single, massive table called `COMPANY_ALL_DATA`. Every time an event occurs (a payroll is generated, an access log is registered, or a leave is requested), all data is duplicated.

Universal Table Schema (`COMPANY_ALL_DATA`)

Column Name	Data Type	Attribute Description / Origin
`employee_id`	`INT`	Employee unique identifier
`employee_name`	`VARCHAR`	Full name of the employee
`employee_email`	`VARCHAR`	Corporate email address
`current_salary`	`DECIMAL`	Current active base salary
`department_id`	`INT`	Department unique identifier
`department_name`	`VARCHAR`	Name of the department
`schedule_id`	`INT`	Work schedule unique identifier
`shift_name`	`VARCHAR`	Shift description (e.g., "Morning", "Night")
`start_time`	`TIME`	Shift start time
`end_time`	`TIME`	Shift end time
`history_id`	`INT`	Salary change record identifier
`new_salary`	`DECIMAL`	Historical data: Updated salary amount
`salary_change_date`	`DATE`	Historical data: Date when the salary changed
`log_id`	`INT`	Access log unique identifier
`access_log_date`	`DATETIME`	Access control: Exact timestamp of entry/exit
`access_event_type`	`VARCHAR`	Access control: Event type (e.g., "Check-In", "Check-Out")
`leave_id`	`INT`	Leave request unique identifier
`leave_type`	`VARCHAR`	Leave details: Type (e.g., "Medical", "Vacation")
`leave_start_date`	`DATE`	Leave details: Start date of the period
`leave_end_date`	`DATE`	Leave details: End date of the period
`leave_status`	`VARCHAR`	Leave details: Current status ("Approved", "Pending")
`payslip_id`	`INT`	Payslip unique identifier
`period_month`	`VARCHAR`	Payroll details: Paid month
`period_year`	`INT`	Payroll details: Paid year
`net_salary_paid`	`DECIMAL`	Payroll details: Final net amount transferred

Anomalies & Problems in 0NF:

- **Massive Redundancy:** If Employee #1 has 50 access logs, their name, email, salary, department, and schedule are written 50 times.
- **Insertion Anomaly:** You cannot create a new department or a new work schedule unless you hire at least one employee to assign it to.
- **Deletion Anomaly:** If you delete the only employee belonging to a specific department, that department disappears from the system entirely.

---

1. First Normal Form (1NF)

Rule to satisfy:

1. All attributes must be **atomic** (a single value per cell, no arrays or comma-separated lists).
2. There must be no **repeating groups** across columns.
3. A Primary Key (PK) must be defined.

Action taken:

To ensure every single event row is uniquely identified without multi-valued cells, we must establish a massive **Composite Primary Key**:

- **Composite PK:** `{employee_id, history_id, log_id, leave_id, payslip_id}`

1NF Schema:

The table structure remains the same as the 0NF table layout above, but we officially declare this composite primary key.

- **Status: Achieved 1NF**. Cells contain single values, but data redundancy and operational anomalies remain exactly the same.

---

2. Second Normal Form (2NF)

Rule to satisfy:

1. Must already be in **1NF**.
2. **No Partial Dependencies:** Any non-key attribute must depend on the *entire* primary key, not just a part of it.

Action taken:

We look at our massive composite key `{employee_id, history_id, log_id, leave_id, payslip_id}` and notice that the non-key fields only care about *one* part of the key:

- `employee_name` only depends on `employee_id`.
- `access_log_date` only depends on `log_id`.
- `net_salary_paid` only depends on `payslip_id`.

We split the universal table into **5 independent groups** according to what actually determines them:

2NF Resulting Tables:

1. Employees (Temporary Base Table)

- **PK:** `employee_id`

- **Attributes:** `employee_name`, `employee_email`, `current_salary`, `department_id`, `department_name`, `schedule_id`, `shift_name`, `start_time`, `end_time`

2. Salary History

- **PK:** `history_id`
- **Attributes:** `employee_id` (FK), `new_salary`, `salary_change_date`

3. Access Logs

- **PK:** `log_id`
- **Attributes:** `employee_id` (FK), `access_log_date`, `access_event_type`

4. Leave Requests

- **PK:** `leave_id`
- **Attributes:** `employee_id` (FK), `leave_type`, `leave_start_date`, `leave_end_date`, `leave_status`

5. Payslips

- **PK:** `payslip_id`
- **Attributes:** `employee_id` (FK), `period_month`, `period_year`, `net_salary_paid`
- **Status: Achieved 2NF**. Tables 2, 3, 4, and 5 are completely clean. However, the **Employees** table still has internal issues.

---

3. Third Normal Form (3NF)

Rule to satisfy:

1. Must already be in **2NF**.
2. **No Transitive Dependencies:** Non-key attributes cannot depend on other non-key attributes. ($X \to Y \to Z$ is forbidden).

Action taken:

Let's analyze the **Employees** table from 2NF:

- `employee_id` determines `department_id`, and `department_id` determines `department_name`.
- `employee_id` determines `schedule_id`, and `schedule_id` determines `shift_name`, `start_time`, `end_time`.

Since `department_name` and schedule details depend on other non-key IDs, we extract them into their own tables.

3NF Resulting Tables:

Table: DEPARTMENTS

- **PK:** `department_id`
- **Attributes:** `department_name`

Table: SCHEDULES

- **PK:** `schedule_id`
- **Attributes:** `shift_name`, `start_time`, `end_time`

Table: EMPLOYEES (Cleaned)

- **PK:** `employee_id`
- **Attributes:** `employee_name`, `employee_email`, `current_salary`, `department_id` (FK), `schedule_id` (FK)

(Note: The Salary History, Access Logs, Leave Requests, and Payslips tables carry over from 2NF completely unchanged).

- **Status: Achieved 3NF**. Transitive dependencies are removed.

---

4. Boyce-Codd Normal Form (BCNF)

Rule to satisfy:

1. Must already be in **3NF**.
2. For every functional dependency $X \to Y$, X **must be a superkey** (a primary key or a candidate key).

Evaluation:

Let's audit our final tables to check if any non-key attribute is acting as a determinant:

- **In `EMPLOYEES`:** * `employee_id` $\to$ all other attributes (`employee_id` is a superkey).
  - `employee_email` $\to$ `employee_id` (Since emails are unique, it serves as a **Candidate Key**. Because the determinant is a key, it complies with BCNF).
- **In all other tables (`DEPARTMENTS`, `SCHEDULES`, etc.):** * The only determinants are the individual single-column Primary Keys (`id`).

Final Verdict:

Because every single functional dependency in this database schema is determined strictly by a valid key, **the design successfully achieves BCNF.**